Indian Statistical Institute

M.Tech.(CS), 2025-2026

Lab Assignment 1

Subject: Operating Systems

Total: 5 marks (ONLY for shell, warmp exercises to be done in class) Deadline: March 12, 2026

1. *Warmup0*: Write a program that forks a child process using the fork system call. The child should print "I am child", followed by its PID, and exit. The parent, after forking, must print "I am parent", followed by its PID. The parent must also reap the dead child before exiting. Run this program a few times. What is the order in which the statements are printed? Can you ensure that the parent always prints after the child by a suitable use of the wait system call?

2. *Warmup1*: Write a program that uses the exec system call to run the "ls -l" command in the program. Running your program should produce the same output as that produced by the "ls -l" command. You must write three versions of your program using the following three different variants of the exec command: execl (takes arguments as a list), execlp (takes arguments as a list, and searches system PATH for the executable), and execvp (takes arguments as a vector, and searches system PATH for the executable). Hint: Example code to invoke these different variants of exec is given below.

```
execl("/bin/ls", "ls", "-l", (char *)NULL);
execlp("ls", "ls", "-l", (char *)NULL);
char *args[] = {"ls", "-l", NULL};
execvp("ls", args);
```

3. *Warmup2*: Write a program `execCmd.c` that executes a simple Linux command with a single argument, for example, sleep 10. Your program should take two command-line arguments: the name of the command and the argument to the command. The program must then fork a child process, and exec the command in the child process. The parent process must wait for the child to finish, reap it, and print a message that the command executed successfully. Your program must throw an error if it is given an incorrect number of arguments. You can choose a suitable variant of exec. Hint: In a main function defined as int main(int argc, char *argv[]), the variable argc contains the number of arguments, and you can access the char * string command-line arguments using the variables argv[0], argv[1], and so on. A sample execution of the program is shown below.

```
$gcc execCmd.c -o execCmd
$./execCmd echo hello
hello
Command successfully completed
$ ./execCmd ls
Incorrect number of arguments
```

4. *Actual assignmnent:* Now build a simple shell to execute user commands, much like the `bash` shell in Linux. This lab will deepen your understanding of various concepts of process management in Linux. A shell takes user input, forks a child process, executes the command using `exec`, waits for the child to finish, and then prompts for the next command. Your shell must execute commands such as:

   ```
   ls
   cat
   echo
   sleep
   ```

   These executables already exist in Linux; your shell simply invokes them using `exec`. You must implement this functionality using `fork`, `exec`, and `wait`. Do **not** use library functions such as `system()`.

Basic requirements

- The shell prompt must be:

  ```
  $<YOUR NAME>
  ```

- The shell runs indefinitely until terminated by `Ctrl+C`.
- Commands are separated by spaces.
- Input limits:
  - Maximum command length: 1024 characters
  - Maximum tokens: 64
  - Maximum token length: 64 characters

You are provided starter code `my_shell.c` that reads input and tokenizes it. The token array contains the command and arguments and can be passed directly to `execvp`.

Error handling

- Empty command → show prompt again
- Invalid command → `exec` fails → print error
- Incorrect arguments → allow executable to print error

Use the `ps` command in another terminal to ensure that no zombie processes remain.

Additional features

- Modify the prompt to show the current working directory:

  ```
  /home/user $
  ```

  Use:

```
getcwd(char *buf, size_t size)
```

- Implement the `cd` command using `chdir`.

  Examples:

  ```
  cd directoryname
  cd ..
  ```

  Note: `chdir` must be executed by the shell itself, not a child process.
- If `exec` fails, the child should exit with:

  ```
  exit(1)
  ```

  The parent should print:

  ```
  EXITSTATUS: 1
  ```

  You can obtain exit status using:

  ```
  wait(&ws);
  WEXITSTATUS(ws)
  ```

Background Execution

If a command ends with `&`, it must run in the background.

Example:

```
sleep 10 &
```

Requirements:

- Shell must not wait for the command to finish
- Shell immediately prompts for next command
- Maximum 64 background processes

To reap background processes without blocking:

```
waitpid(-1, NULL, WNOHANG)
```

When a background process finishes, print:

```
Shell: Background process finished
```

The exit Command

Implement:

`exit`

When invoked:

- Terminate all background processes using `kill`
- Clean up memory
- Exit the shell

Obviously, if the shell is receiving the command to exit, it goes without saying that it will not have any active foreground process running. Before exiting, the shell must also clean up any internal state (e.g., free dynamically allocated memory), and terminate in a clean manner.

Handling Ctrl+C

Modify your shell so that:

- `Ctrl+C` does **not terminate the shell**
- It terminates only the foreground process
- Background processes continue running

Use custom signal handlers with:

```
signal()
sigaction()
```

Serial and Parallel Foreground Execution

Support multiple commands.

Serial execution

Commands separated by:

`&&`

Example:

```
sleep 2 && ls && echo done
```

Commands execute sequentially.

Parallel execution

Commands separated by:

`&&&`

Example:

```
sleep 5 &&& sleep 6 &&& sleep 7
```

Commands run simultaneously.

Shell returns to prompt only after all commands finish.

Ctrl+C behavior

- In parallel mode → terminate all foreground processes
- In serial mode → terminate current command and cancel remaining commands

Use long running commands such as `sleep` to test serial and parallel execution.

You can take the following simple Shell code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/wait.h>

#define LINE_SIZE 1024

int main ( )
{
   char line[LINE_SIZE];

   while (1) {
      /* Read a line of input from the user */
      printf("$ "); fgets(line,LINE_SIZE,stdin); line[strlen(line)-1] = '\0';

      /* Nothing to do for a blank line */
      if (!strcmp(line,"")) continue;
```

```
      /* Terminate the shell */
      if (!strcmp(line,"exit")) break;

      if (fork()) {  /* Parent process */
         wait(NULL); /* Wait until the child terminates */
      } else {       /* Child process */
         /* WRITE CODE HERE */
      }
   }

   exit(0);
}
```